

Python Interview Questions

Complete Syllabus - Theory, Concepts & Internals

96 questions with answers

Interview Preparation Guide

Python Basics & Data Types

Q1. What is Python and what are its key features?

Python is a high-level, interpreted, dynamically-typed, garbage-collected language. Key features: easy/readable syntax, multi-paradigm (OOP, procedural, functional), extensive standard library, portable, dynamically typed, and supports automatic memory management.

Q2. Is Python compiled or interpreted?

Python source is first compiled to bytecode (.pyc) and then executed by the Python Virtual Machine (PVM). So it is both compiled (to bytecode) and interpreted (bytecode is interpreted).

Q3. What are the built-in data types in Python?

Numeric (int, float, complex), Sequence (list, tuple, range, str), Set (set, frozenset), Mapping (dict), Boolean (bool), Binary (bytes, bytearray, memoryview), and NoneType.

Q4. What is the difference between a list and a tuple?

Lists are mutable, defined with [], slightly slower, used for homogeneous changing data. Tuples are immutable, defined with (), faster, hashable (can be dict keys/set members), used for fixed collections.

Q5. What is the difference between is and ==?

'==' compares values (equality). 'is' compares identity (whether two references point to the same object in memory).

Q6. What is the difference between mutable and immutable objects?

Mutable objects can be changed after creation (list, dict, set, bytearray). Immutable objects cannot (int, float, str, tuple, frozenset, bytes). Immutables are hashable.

Q7. How is memory managed in Python?

Python uses a private heap managed by the interpreter, reference counting for deallocation, and a cyclic garbage collector to clean up reference cycles. The pymalloc allocator manages small objects.

Q8. What is the difference between deep copy and shallow copy?

A shallow copy (copy.copy) copies the outer object but shares references to nested objects. A deep copy (copy.deepcopy) recursively copies everything, producing fully independent objects.

Q9. What are Python's numeric types and integer behavior?

int (arbitrary precision, no overflow), float (double precision), complex. Integers grow as large as memory allows.

Q10. What is string interning?

Python caches (interns) small integers (-5 to 256) and some short strings so identical literals share the same object, saving memory and speeding up comparisons.

Q11. What is the difference between str, bytes, and bytearray?

str is an immutable sequence of Unicode characters; bytes is an immutable sequence of raw bytes; bytearray is a mutable sequence of bytes.

Q12. How do you format strings in Python?

Using f-strings (f'{x}'), str.format(), %-formatting, or string.Template. f-strings (3.6+) are the most readable and fastest.

Operators, Control Flow & Loops

Q13. What is the difference between / and // operators?

'/' is true division returning a float; '//' is floor division returning the largest integer less than or equal to the result.

Q14. What does the ** operator do?

Exponentiation ($a ** b = a$ to the power b). In function calls/definitions ****** unpacks or collects keyword arguments.

Q15. What are the membership and identity operators?

Membership: `in`, `not in` (test presence in a sequence). Identity: `is`, `is not` (test object identity).

Q16. Does Python have a switch/case statement?

Before 3.10 no; people used dict dispatch or `if/elif`. Python 3.10+ has structural pattern matching via `match/case`.

Q17. What is the difference between break, continue, and pass?

`break` exits the nearest loop; `continue` skips to the next iteration; `pass` is a no-op placeholder.

Q18. What is the else clause on loops?

A `for/while` loop's `else` block runs when the loop finishes normally (no `break`). Useful for search loops to detect 'not found'.

Q19. What is short-circuit evaluation?

Logical operators stop evaluating as soon as the result is known: `'and'` returns the first falsy operand, `'or'` returns the first truthy operand.

Q20. What is a ternary (conditional) expression in Python?

`value_if_true if condition else value_if_false`.

Q21. What is the walrus operator :=?

Introduced in 3.8, it assigns and returns a value within an expression, e.g. `while (line := f.readline()):`

Functions & Scope

Q22. What is the difference between `*args` and `**kwargs`?

`*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict.

Q23. What are default arguments and the mutable default argument pitfall?

Defaults are evaluated once at definition time. Using a mutable default (`def f(x=[])`) shares the same object across calls. Use `None` and create inside the function instead.

Q24. What is a lambda function?

An anonymous, single-expression function: `lambda x: x + 1`. Useful for short callbacks like `key=` in `sorted()`.

Q25. What is the difference between positional and keyword arguments?

Positional are matched by order; keyword by name. Python also supports positional-only (`/`) and keyword-only (`*`) parameter markers.

Q26. Explain Python's LEGB scope rule.

Name lookup order: Local, Enclosing (nonlocal functions), Global (module), Built-in.

Q27. What do the `global` and `nonlocal` keywords do?

`global` rebinds a name at module level from inside a function; `nonlocal` rebinds a name in the nearest enclosing (non-global) function scope.

Q28. What is a closure?

A nested function that captures and remembers variables from its enclosing scope even after that scope has finished executing.

Q29. Are arguments passed by value or by reference in Python?

Neither exactly; Python passes object references by value ('pass by assignment'). Mutating a mutable argument affects the caller; rebinding the parameter does not.

Q30. What is a first-class function?

Functions are objects: they can be assigned to variables, passed as arguments, returned, and stored in data structures.

Q31. What is recursion and what is the recursion limit?

A function calling itself. Python caps recursion (default ~1000) via `sys.getrecursionlimit()` to avoid stack overflow; adjustable with `sys.setrecursionlimit()`.

Object-Oriented Programming

Q32. What is the difference between a class and an object?

A class is a blueprint defining attributes and methods; an object is an instance of a class with its own state.

Q33. What is the `__init__` method?

The initializer (constructor-like) method called automatically when an object is created, used to set up instance attributes. The actual constructor is `__new__`.

Q34. What is the difference between `__new__` and `__init__`?

`__new__` creates and returns the new instance (allocates it); `__init__` initializes the already created instance. `__new__` is used for immutables and singletons.

Q35. What is `self`?

The conventional name for the instance reference passed implicitly as the first argument to instance methods.

Q36. What are instance, class, and static methods?

Instance methods take `self`; class methods take `cls` and use `@classmethod` (operate on the class); static methods use `@staticmethod` and take neither (plain utility functions in the class namespace).

Q37. What is inheritance and what types does Python support?

Inheritance lets a class reuse another's members. Python supports single, multiple, multilevel, hierarchical, and hybrid inheritance.

Q38. What is the MRO (Method Resolution Order)?

The order Python searches base classes for a method, computed via the C3 linearization algorithm. View it with `ClassName.__mro__` or `ClassName.mro()`.

Q39. What is method overriding vs overloading?

Overriding: a subclass redefines a parent method. Overloading (multiple signatures) is not natively supported; emulated via default args, `*args`, or `functools.singledispatch`.

Q40. What is encapsulation and how is it done in Python?

Bundling data and methods, restricting access. Single underscore `_x` is a convention (protected); double underscore `__x` triggers name mangling (pseudo-private).

Q41. What is polymorphism?

The ability for different types to respond to the same interface/operation, e.g. `len()` works on lists and strings, or duck typing where behavior matters more than type.

Q42. What is abstraction and how do abstract classes work?

Hiding implementation behind a clean interface. Use the abc module: subclass ABC and decorate methods with @abstractmethod so the class cannot be instantiated until they are implemented.

Q43. What are dunder/magic methods?

Special methods with double underscores (`__str__`, `__repr__`, `__len__`, `__eq__`, `__add__`) that integrate objects with built-in operations and operators.

Q44. What is the difference between `__str__` and `__repr__`?

`__str__` gives a readable, user-facing string; `__repr__` gives an unambiguous, developer-facing representation (ideally reconstructable). `repr()` is the fallback for `str()`.

Q45. What are @property and getters/setters?

@property turns a method into a managed attribute, allowing computed values and validation while keeping attribute-style access. `@x.setter` defines the assignment behavior.

Q46. What is super() used for?

To call methods of a parent/next class in the MRO, commonly `super().__init__()` to invoke base initialization, enabling cooperative multiple inheritance.

Q47. What are `__slots__`?

A class attribute that declares a fixed set of instance attributes, preventing a per-instance `__dict__` to save memory and slightly speed up attribute access.

Q48. What is a dataclass?

A class decorated with @dataclass (3.7+) that auto-generates `__init__`, `__repr__`, `__eq__`, etc. from typed field declarations, reducing boilerplate.

Q49. What is a metaclass?

The class of a class; it controls class creation. 'type' is the default metaclass. Used for frameworks (ORMs, validation) to customize class construction.

Iterators, Generators & Comprehensions

Q50. What is the difference between an iterable and an iterator?

An iterable implements `__iter__` and can produce an iterator (list, str). An iterator implements `__iter__` and `__next__` and yields values one at a time, tracking state.

Q51. What is a generator?

A function using `yield` that returns a lazy iterator, producing values on demand and preserving state between calls -- memory efficient for large/streaming data.

Q52. What is the difference between `yield` and `return`?

`return` ends the function and sends back one value; `yield` pauses the function, emits a value, and resumes on the next iteration.

Q53. What is a generator expression?

A lazy comprehension using parentheses: `(x*x for x in range(10))`. Produces items one at a time instead of building a full list.

Q54. What are list, dict, and set comprehensions?

Concise constructs to build collections: `[x for x in it]`, `{k: v for ...}`, `{x for x in it}`. They are faster and more readable than equivalent loops.

Q55. What does the `yield from` statement do?

Delegates iteration to a sub-generator/iterable, yielding all its values and transparently passing through sent values and return values.

Q56. How does the `iter()` and `next()` pair work?

`iter(obj)` returns an iterator; `next(iterator)` fetches the next value and raises `StopIteration` when exhausted. `next(it, default)` returns default instead of raising.

Q57. What are the advantages of generators over lists?

Lazy evaluation, constant/low memory footprint, ability to represent infinite sequences, and pipeline-friendly composition.

Decorators, Closures & Functional Tools

Q58. What is a decorator?

A callable that takes a function/class and returns a modified version, applied with `@`. Used for logging, timing, caching, access control, etc.

Q59. Why use `functools.wraps` inside a decorator?

It copies the original function's metadata (`__name__`, `__doc__`) to the wrapper so introspection and debugging still reflect the wrapped function.

Q60. What is `functools.lru_cache`?

A decorator that memoizes function results based on arguments, caching up to `maxsize` entries to avoid recomputation.

Q61. What do `map`, `filter`, and `reduce` do?

`map` applies a function to each item; `filter` keeps items where a predicate is true; `functools.reduce` folds a sequence into a single value using a binary function.

Q62. What is the difference between a decorator with and without arguments?

A plain decorator wraps a function directly. A parameterized decorator is a factory: an outer function takes the arguments and returns the actual decorator.

Q63. What is partial application (`functools.partial`)?

Creating a new callable with some arguments pre-filled, e.g. `partial(int, base=2)` for binary parsing.

Exception Handling

Q64. How does try/except/else/finally work?

try runs risky code; except handles matching exceptions; else runs if no exception occurred; finally always runs (cleanup), even on return or exception.

Q65. What is the difference between an error and an exception?

Errors (e.g. `SyntaxError`, or unrecoverable issues) often can't be handled at runtime; exceptions are runtime events that can be caught and handled. In Python both derive from `BaseException`.

Q66. How do you raise and re-raise exceptions?

Use `'raise SomeError(msg)'` to raise, bare `'raise'` to re-raise the current exception, and `'raise X from Y'` to chain causes.

Q67. How do you create a custom exception?

Subclass `Exception` (or a more specific built-in): `class MyError(Exception): pass`, then raise it.

Q68. What is the purpose of the finally block?

To guarantee cleanup code (closing files, releasing locks) runs whether or not an exception was raised.

Q69. What is exception chaining?

When handling one exception raises another, Python links them via `__cause__` (explicit, `raise from`) or `__context__` (implicit), preserving the original traceback.

Q70. What are some common built-in exceptions?

`ValueError`, `TypeError`, `KeyError`, `IndexError`, `AttributeError`, `FileNotFoundError`, `ZeroDivisionError`, `StopIteration`, `ImportError`.

Modules, Packages & Files

Q71. What is the difference between a module and a package?

A module is a single .py file; a package is a directory of modules (historically with `__init__.py`) that namespaces them.

Q72. What is the difference between `import x` and `from x import y`?

`import x` brings in the whole module (access as `x.y`); `from x import y` brings a specific name into the current namespace.

Q73. What does `if __name__ == '__main__':` do?

It guards code so it runs only when the file is executed directly, not when imported as a module.

Q74. What is the `PYTHONPATH` and `sys.path`?

`sys.path` is the list of directories Python searches for imports; `PYTHONPATH` is an environment variable that prepends additional directories to it.

Q75. What are the file modes in Python?

'r' read, 'w' write/truncate, 'a' append, 'x' exclusive create, 'b' binary, 't' text, '+' read/write.

Q76. Why use a `with` statement (context manager) for files?

It guarantees the file is closed automatically even if an exception occurs, via `__enter__` and `__exit__`.

Q77. How do you create a custom context manager?

Implement `__enter__`/`__exit__` in a class, or use the `@contextlib.contextmanager` decorator on a generator that yields once.

Q78. What is a virtual environment and why use one?

An isolated Python environment (`venv`) with its own packages, so projects don't share/conflict on dependency versions.

Q79. What is `pip` and `requirements.txt`?

`pip` is Python's package installer; `requirements.txt` pins project dependencies for reproducible installs via `'pip install -r'`.

Advanced & Internals

Q80. What is the GIL (Global Interpreter Lock)?

A mutex in CPython that allows only one thread to execute Python bytecode at a time, simplifying memory management but limiting CPU-bound multithreading.

Q81. How do you achieve true parallelism in Python despite the GIL?

Use multiprocessing (separate processes/interpreters), C extensions that release the GIL, or async/native libraries. Threads still help with I/O-bound work.

Q82. What is the difference between multithreading and multiprocessing?

Threads share memory and are good for I/O-bound tasks but limited by the GIL for CPU work; processes have separate memory and achieve true CPU parallelism at higher overhead.

Q83. What is asyncio / async-await?

A single-threaded concurrency model using an event loop and coroutines (async def / await) to handle many I/O-bound tasks cooperatively without blocking.

Q84. What is the difference between concurrency and parallelism?

Concurrency is structuring a program to handle many tasks (interleaved progress); parallelism is actually executing multiple tasks simultaneously on multiple cores.

Q85. How does garbage collection work in CPython?

Primarily reference counting (objects freed when count hits zero), plus a generational cyclic garbage collector (gc module) to reclaim reference cycles.

Q86. What are Python's *.pyc files?

Cached compiled bytecode stored in `__pycache__` to speed up subsequent imports by skipping recompilation.

Q87. What is monkey patching?

Dynamically modifying or replacing a class/module attribute at runtime, often for testing or patching third-party behavior.

Q88. What is duck typing?

An object's suitability is determined by the presence of methods/attributes rather than its explicit type: 'if it walks like a duck...'

Q89. What are type hints and are they enforced?

Optional annotations (def f(x: int) -> str) for readability and static checkers like mypy. They are NOT enforced at runtime by the interpreter.

Q90. What is the difference between @staticmethod and a module-level function?

Functionally similar, but a staticmethod lives in the class namespace for logical grouping and is accessible via the class/instance.

Q91. What does the zip() function do?

Aggregates elements from multiple iterables into tuples pairwise, stopping at the shortest; zip(*matrix) transposes.

Q92. What is the difference between sort() and sorted()?

list.sort() sorts in place and returns None; sorted() returns a new sorted list and works on any iterable.

Q93. What does enumerate() do?

Yields (index, value) pairs while iterating, with an optional start index.

Q94. What is the difference between any() and all()?

any() returns True if at least one element is truthy; all() returns True only if every element is truthy (and True for empty iterables).

Q95. What are *args unpacking and dictionary unpacking in calls?

func(*list) spreads a sequence into positional args; func(**dict) spreads a mapping into keyword args. Also used to merge: {**a, **b}.

Q96. What is the difference between shallow and deep equality for collections?

== compares element-by-element recursively (value equality); identity (is) checks the same object. Nested mutable contents are compared by value with ==.